

```
In [94]: import seaborn as sns
import pandas as pd

# Load the Titanic dataset
df = sns.load_dataset("titanic")
df
# skewness
```

Out[94]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
886	0	2	male	27.0	0	0	13.0000	S	Second	man	True	NaN	Southampton	no	True
887	1	1	female	19.0	0	0	30.0000	S	First	woman	False	B	Southampton	yes	True
888	0	3	female	NaN	1	2	23.4500	S	Third	woman	False	NaN	Southampton	no	False
889	1	1	male	26.0	0	0	30.0000	C	First	man	True	C	Cherbourg	yes	True
890	0	3	male	32.0	0	0	7.7500	Q	Third	man	True	NaN	Queenstown	no	True

891 rows × 15 columns

```
In [99]: df.skew()  
#info(), desc, missing , outliers
```

C:\Users\HP\AppData\Local\Temp\ipykernel_9652\423298799.py:1: FutureWarning: Dropping of nuisance columns in DataFrame reductions (with 'numeric_only=None') is deprecated; in a future version this will raise TypeError. Select only valid columns before calling the reduction.

```
df.skew()
```

```
Out[99]: survived      0.478523  
pclass      -0.630548  
age         0.389108  
sibsp       3.695352  
parch       2.749117  
fare        4.787317  
adult_male  -0.420431  
alone       -0.420431  
dtype: float64
```

```
In [95]: df.describe()
```

```
Out[95]:
```

	survived	pclass	age	sibsp	parch	fare
count	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

```
In [4]: df.shape#rows, columns
```

```
Out[4]: (891, 15)
```

```
In [5]: df.isnull().sum()
```

```
Out[5]: survived      0
        pclass        0
        sex           0
        age          177
        sibsp         0
        parch         0
        fare          0
        embarked      2
        class         0
        who           0
        adult_male     0
        deck          688
        embark_town     2
        alive          0
        alone          0
        dtype: int64
```

```
In [6]: df.duplicated()
```

```
Out[6]: 0      False
        1      False
        2      False
        3      False
        4      False
        ...
        886    True
        887    False
        888    False
        889    False
        890    False
        Length: 891, dtype: bool
```

```
In [7]: df.duplicated().sum()
```

```
Out[7]: 107
```

```
In [8]: df1=df.drop_duplicates()
```

```
In [9]: df1.shape
```

```
Out[9]: (784, 15)
```

```
In [10]: df1.isnull().sum()
```

```
Out[10]: survived      0  
pclass      0  
sex          0  
age        106  
sibsp       0  
parch       0  
fare        0  
embarked     2  
class        0  
who          0  
adult_male   0  
deck       582  
embark_town   2  
alive        0  
alone        0  
dtype: int64
```

```
In [11]: 784
```

```
Out[11]: 784
```

```
In [12]: (df1.isnull().sum()/df1.shape[0])*100
```

```
Out[12]: survived      0.000000  
pclass      0.000000  
sex         0.000000  
age        13.520408  
sibsp      0.000000  
parch      0.000000  
fare       0.000000  
embarked    0.255102  
class      0.000000  
who         0.000000  
adult_male  0.000000  
deck       74.234694  
embark_town 0.255102  
alive      0.000000  
alone      0.000000  
dtype: float64
```

```
In [13]: s=(df1.isnull().sum()/df1.shape[0])*100  
round(s,2)
```

```
Out[13]: survived      0.00  
pclass      0.00  
sex         0.00  
age        13.52  
sibsp      0.00  
parch      0.00  
fare       0.00  
embarked    0.26  
class      0.00  
who         0.00  
adult_male  0.00  
deck       74.23  
embark_town 0.26  
alive      0.00  
alone      0.00  
dtype: float64
```

```
In [14]: round((df1.isnull().sum()/df1.shape[0])*100,2)
```

```
Out[14]: survived      0.00  
pclass      0.00  
sex         0.00  
age        13.52  
sibsp      0.00  
parch      0.00  
fare       0.00  
embarked   0.26  
class      0.00  
who        0.00  
adult_male 0.00  
deck       74.23  
embark_town 0.26  
alive      0.00  
alone      0.00  
dtype: float64
```

```
In [15]: df1.dropna
```

```
Out[15]: <bound method DataFrame.dropna of      survived  pclass    sex  age  sibsp  parch    fare embarked  class
\
0          0      3   male  22.0    1     0   7.2500      S   Third
1          1      1  female  38.0    1     0  71.2833      C   First
2          1      3  female  26.0    0     0   7.9250      S   Third
3          1      1  female  35.0    1     0  53.1000      S   First
4          0      3   male  35.0    0     0   8.0500      S   Third
..      ...    ...    ...    ...    ...    ...    ...    ...    ...
885         0      3  female  39.0    0     5  29.1250      Q   Third
887         1      1  female  19.0    0     0  30.0000      S   First
888         0      3  female   NaN    1     2  23.4500      S   Third
889         1      1   male  26.0    0     0  30.0000      C   First
890         0      3   male  32.0    0     0   7.7500      Q   Third

      who  adult_male  deck  embark_town  alive  alone
0     man         True  NaN  Southampton    no  False
1  woman        False    C   Cherbourg   yes  False
2  woman        False  NaN  Southampton   yes   True
3  woman        False    C   Southampton   yes  False
4     man         True  NaN  Southampton    no   True
..     ...         ...    ...         ...    ...    ...
885  woman        False  NaN   Queenstown    no  False
887  woman        False    B   Southampton   yes   True
888  woman        False  NaN   Southampton    no  False
889    man         True    C   Cherbourg   yes   True
890    man         True  NaN   Queenstown    no   True

[784 rows x 15 columns]>
```

As we can see from the above result that deck has 74% null values and Age has 13.52% and Embarked has 0.26% of null values.

So, it's not a good idea to fill 74% of null values. So, we will drop the deck column. Embarked column has only 0.26% of null values so, we drop the null values rows of Embarked column.

```
In [16]: df2 =df1.drop(columns='deck')
df2
```

Out[16]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	Southampton	no	True
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
885	0	3	female	39.0	0	5	29.1250	Q	Third	woman	False	Queenstown	no	False
887	1	1	female	19.0	0	0	30.0000	S	First	woman	False	Southampton	yes	True
888	0	3	female	NaN	1	2	23.4500	S	Third	woman	False	Southampton	no	False
889	1	1	male	26.0	0	0	30.0000	C	First	man	True	Cherbourg	yes	True
890	0	3	male	32.0	0	0	7.7500	Q	Third	man	True	Queenstown	no	True

784 rows × 14 columns

```
In [17]: df1=df1.drop(columns='deck')

#or

# df1.drop(columns='deck',inplace=True)
```



```
In [18]: df2
```

```
Out[18]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	Southampton	no	True
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
885	0	3	female	39.0	0	5	29.1250	Q	Third	woman	False	Queenstown	no	False
887	1	1	female	19.0	0	0	30.0000	S	First	woman	False	Southampton	yes	True
888	0	3	female	NaN	1	2	23.4500	S	Third	woman	False	Southampton	no	False
889	1	1	male	26.0	0	0	30.0000	C	First	man	True	Cherbourg	yes	True
890	0	3	male	32.0	0	0	7.7500	Q	Third	man	True	Queenstown	no	True

784 rows × 14 columns

```
In [19]: df2.dropna(subset=['embarked','embark_town'], axis=0,inplace=True)  
df2.shape
```

```
Out[19]: (782, 14)
```

```
In [20]: df2.isnull().sum()
```

```
Out[20]: survived      0
pclass      0
sex          0
age         106
sibsp       0
parch       0
fare        0
embarked    0
class       0
who         0
adult_male  0
embark_town 0
alive       0
alone       0
dtype: int64
```

3 column missing

1--- 50% > miss--column complete drop 2--- mild 1%< 2 row, delete-no change --rows delete 3-- 13%-- nan--impute(substitute)

how to handle age column? - imputation

Mean imputation is suitable when the data is normally distributed and has no extreme outliers.

Median imputation is preferable when the data contains outliers or is skewed.

```
In [21]: #df2["age"].fillna()
```

```
In [22]: df2.describe()
```

```
Out[22]:
```

	survived	pclass	age	sibsp	parch	fare
count	782.000000	782.000000	676.000000	782.000000	782.000000	782.000000
mean	0.410486	2.246803	29.809793	0.524297	0.416880	34.595913
std	0.492237	0.853828	14.725652	0.987138	0.837728	52.176458
min	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	0.000000	1.000000	20.000000	0.000000	0.000000	8.050000
50%	0.000000	3.000000	28.000000	0.000000	0.000000	15.875000
75%	1.000000	3.000000	39.000000	1.000000	1.000000	33.375000
max	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

```
In [23]: x=df2["age"]  
x
```

```
Out[23]: 0      22.0  
1      38.0  
2      26.0  
3      35.0  
4      35.0  
...  
885    39.0  
887    19.0  
888     NaN  
889    26.0  
890    32.0  
Name: age, Length: 782, dtype: float64
```

```
In [24]: import statistics as st  
st.mean(x)
```

```
Out[24]: nan
```

```
In [25]: st.median(x)
```

```
Out[25]: nan
```

```
In [26]: import numpy as np
```

```
In [27]: np.mean(x)
```

```
Out[27]: 29.809792899408283
```

```
In [28]: np.median(x)
```

```
Out[28]: nan
```

```
In [29]: st.mode(x)
```

```
Out[29]: 24.0
```

```
In [30]: #np.mode(x)
```

```
In [31]: df2["age"].mean()
```

```
Out[31]: 29.809792899408283
```

```
In [32]: df2["age"].median()
```

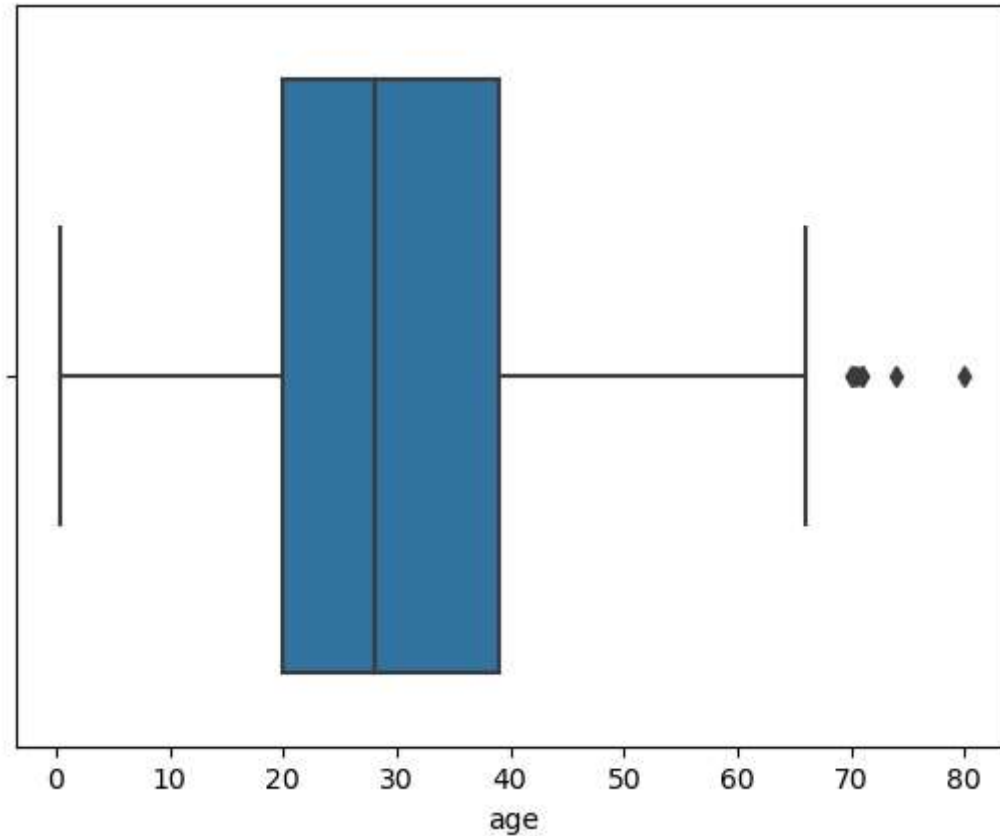
```
Out[32]: 28.0
```

```
In [ ]:
```

```
In [33]: df2["age"]  
sns.boxplot(df2["age"])
```

C:\Users\HP\anaconda3\lib\site-packages\seaborn_decorators.py:36: FutureWarning: Pass the following variable as a keyword arg: x. From version 0.12, the only valid positional argument will be `data`, and passing other arguments without an explicit keyword will result in an error or misinterpretation.
warnings.warn(

```
Out[33]: <AxesSubplot:xlabel='age'>
```



```
In [34]: df2["age"].fillna(df2["age"].median(),inplace=True)
```

```
In [35]: df2.isnull().sum()
```

```
Out[35]: survived      0  
pclass      0  
sex         0  
age         0  
sibsp      0  
parch      0  
fare       0  
embarked    0  
class      0  
who        0  
adult_male  0  
embark_town 0  
alive      0  
alone      0  
dtype: int64
```

```
In [36]: df2["age"].fillna(df2["age"].median(),inplace=True)
```

```
In [37]: df2.isnull().sum()
```

```
Out[37]: survived      0  
pclass      0  
sex         0  
age         0  
sibsp      0  
parch      0  
fare       0  
embarked    0  
class      0  
who        0  
adult_male  0  
embark_town 0  
alive      0  
alone      0  
dtype: int64
```

```
In [38]: df2["age"].mean()
```

```
Out[38]: 29.564475703324806
```

```
In [39]: df2["age"].median()
```

```
Out[39]: 28.0
```

```
In [40]: df2.describe()
```

```
Out[40]:
```

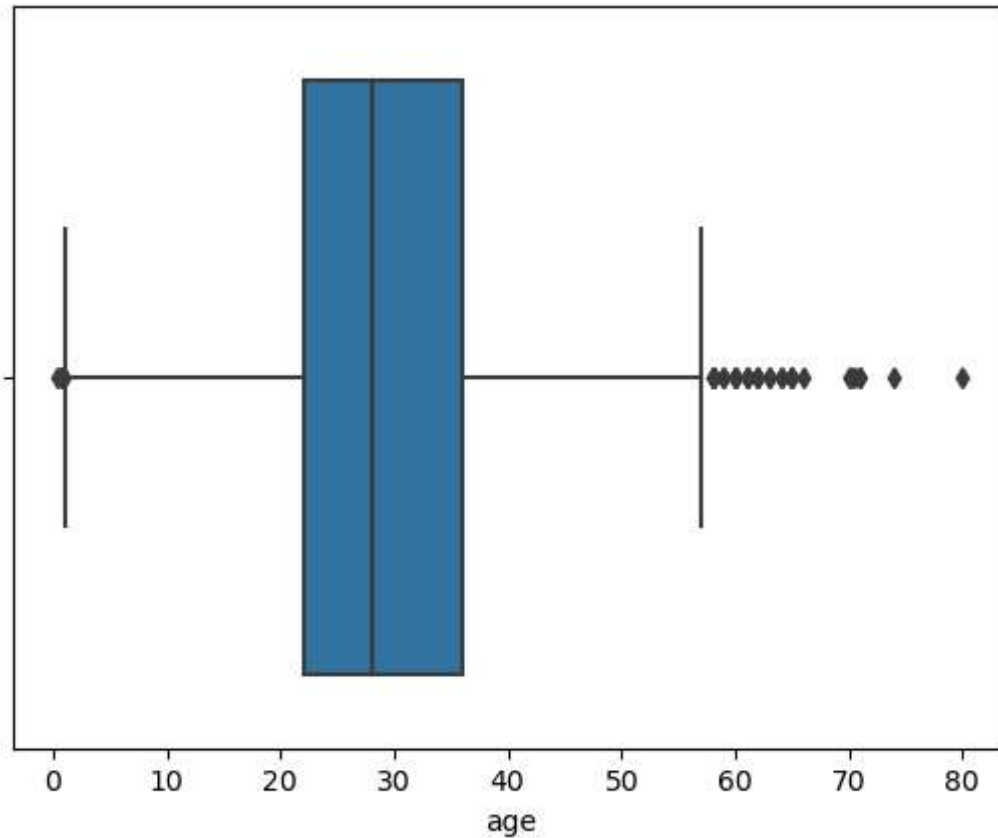
	survived	pclass	age	sibsp	parch	fare
count	782.000000	782.000000	782.000000	782.000000	782.000000	782.000000
mean	0.410486	2.246803	29.564476	0.524297	0.416880	34.595913
std	0.492237	0.853828	13.703948	0.987138	0.837728	52.176458
min	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	0.000000	1.000000	22.000000	0.000000	0.000000	8.050000
50%	0.000000	3.000000	28.000000	0.000000	0.000000	15.875000
75%	1.000000	3.000000	36.000000	1.000000	1.000000	33.375000
max	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

```
In [41]: import seaborn as sns  
import matplotlib.pyplot as plt
```

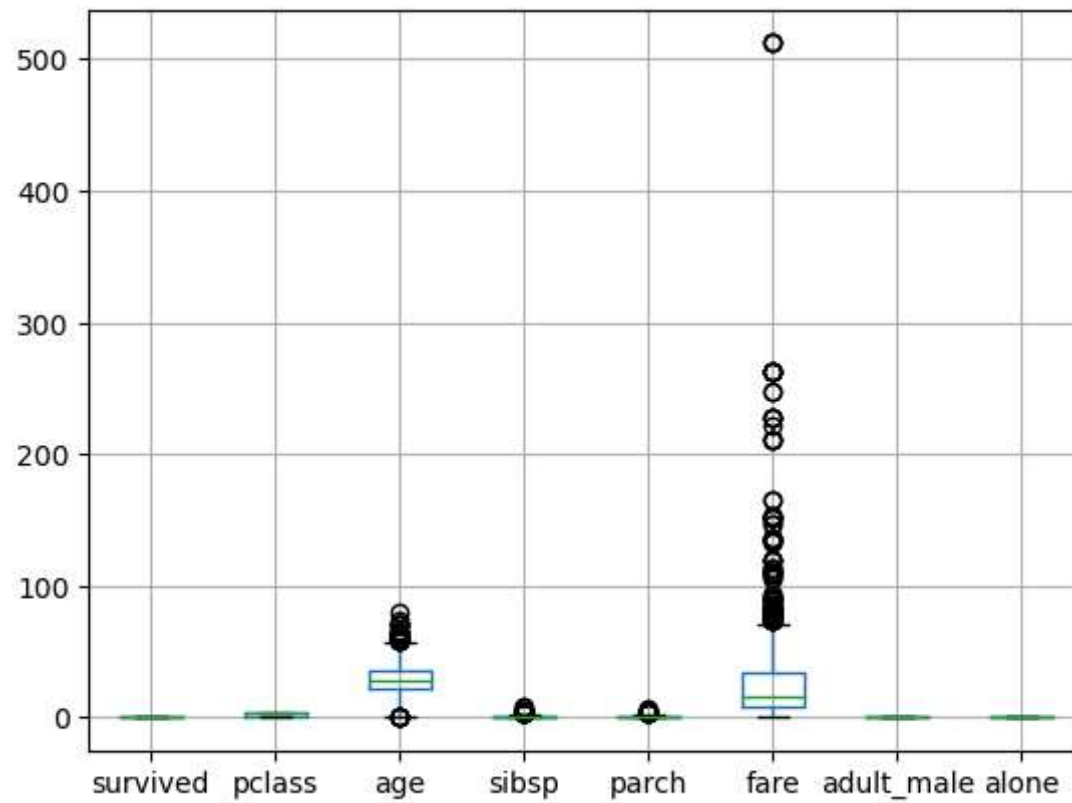
```
In [42]: sns.boxplot(df2["age"])
plt.show()
```

C:\Users\HP\anaconda3\lib\site-packages\seaborn_decorators.py:36: FutureWarning: Pass the following variable as a keyword arg: x. From version 0.12, the only valid positional argument will be `data`, and passing other arguments without an explicit keyword will result in an error or misinterpretation.

warnings.warn(




```
In [43]: df2.boxplot()  
plt.show()
```



```
In [44]: df2.skew()
```

```
C:\Users\HP\AppData\Local\Temp\ipykernel_9652\834593052.py:1: FutureWarning: Dropping of nuisance columns in DataFrame reductions (with 'numeric_only=None') is deprecated; in a future version this will raise TypeError. Select only valid columns before calling the reduction.
```

```
df2.skew()
```

```
Out[44]: survived      0.364636
pclass      -0.495610
age         0.447515
sibsp       3.051088
parch       2.628645
fare        4.583206
adult_male  -0.311181
alone       -0.274151
dtype: float64
```

Symmetrical Distribution: Skewness is around 0.

Moderately Skewed: Skewness between -0.5 and 0.5.

Highly Skewed: Skewness less than -1 or greater than 1.

For skewness values between -0.5 and 0.5, the distribution is considered fairly symmetrical and typically does not indicate a severe skew. Therefore, in this range, outliers are less likely to be a significant problem. (for age)

as outlier has so rle,we can use mean for imputation

```
In [45]: import numpy as np
df2['age'].fillna(np.mean(df2['age'])), inplace=True)
```

In [46]: df2

Out[46]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	Southampton	no	True
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
885	0	3	female	39.0	0	5	29.1250	Q	Third	woman	False	Queenstown	no	False
887	1	1	female	19.0	0	0	30.0000	S	First	woman	False	Southampton	yes	True
888	0	3	female	28.0	1	2	23.4500	S	Third	woman	False	Southampton	no	False
889	1	1	male	26.0	0	0	30.0000	C	First	man	True	Cherbourg	yes	True
890	0	3	male	32.0	0	0	7.7500	Q	Third	man	True	Queenstown	no	True

782 rows × 14 columns

```
In [47]: df2.info()
```

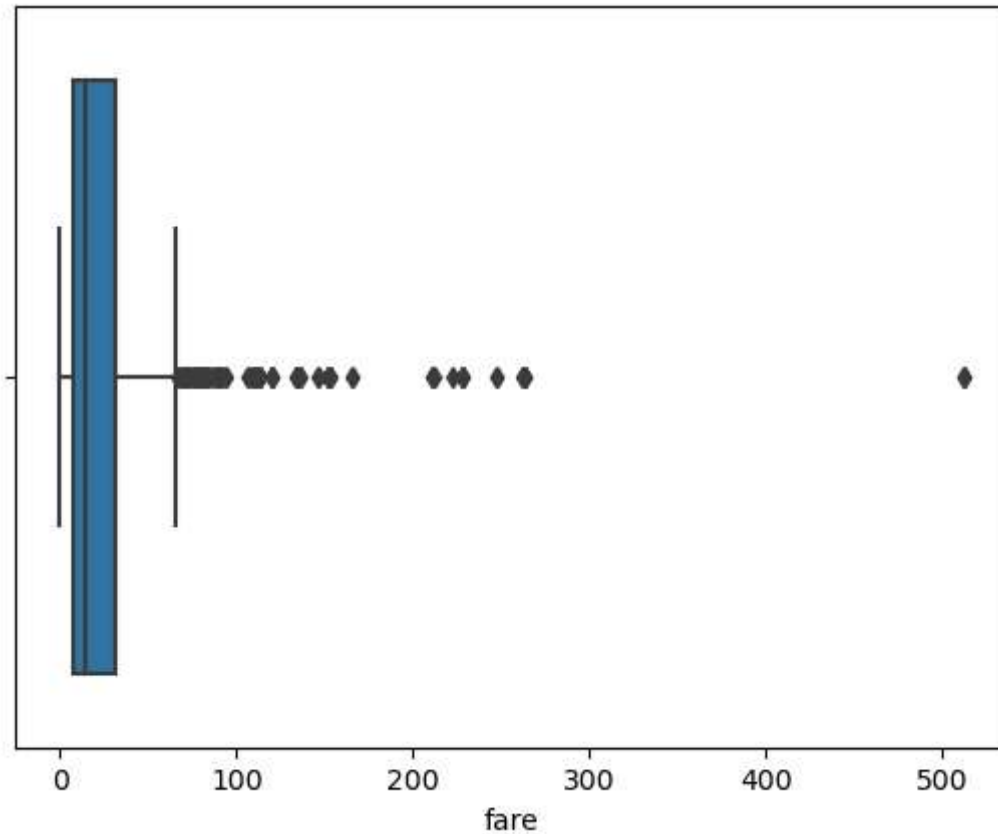
```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 782 entries, 0 to 890
Data columns (total 14 columns):
#   Column      Non-Null Count  Dtype
---  -
0   survived    782 non-null    int64
1   pclass      782 non-null    int64
2   sex         782 non-null    object
3   age         782 non-null    float64
4   sibsp       782 non-null    int64
5   parch       782 non-null    int64
6   fare        782 non-null    float64
7   embarked    782 non-null    object
8   class       782 non-null    category
9   who         782 non-null    object
10  adult_male   782 non-null    bool
11  embark_town  782 non-null    object
12  alive        782 non-null    object
13  alone        782 non-null    bool
dtypes: bool(2), category(1), float64(2), int64(4), object(5)
memory usage: 75.7+ KB
```

 outliers 

```
In [158]: sns.boxplot(df["fare"])  
plt.show()
```

C:\Users\HP\anaconda3\lib\site-packages\seaborn_decorators.py:36: FutureWarning: Pass the following variable as a keyword arg: x. From version 0.12, the only valid positional argument will be `data`, and passing other arguments without an explicit keyword will result in an error or misinterpretation.

warnings.warn(



missing values handled

outlier check

```
In [48]: df2.skew()  
0.5
```

C:\Users\HP\AppData\Local\Temp\ipykernel_9652\834593052.py:1: FutureWarning: Dropping of nuisance columns in DataFrame reductions (with 'numeric_only=None') is deprecated; in a future version this will raise TypeError. Select only valid columns before calling the reduction.

```
df2.skew()
```

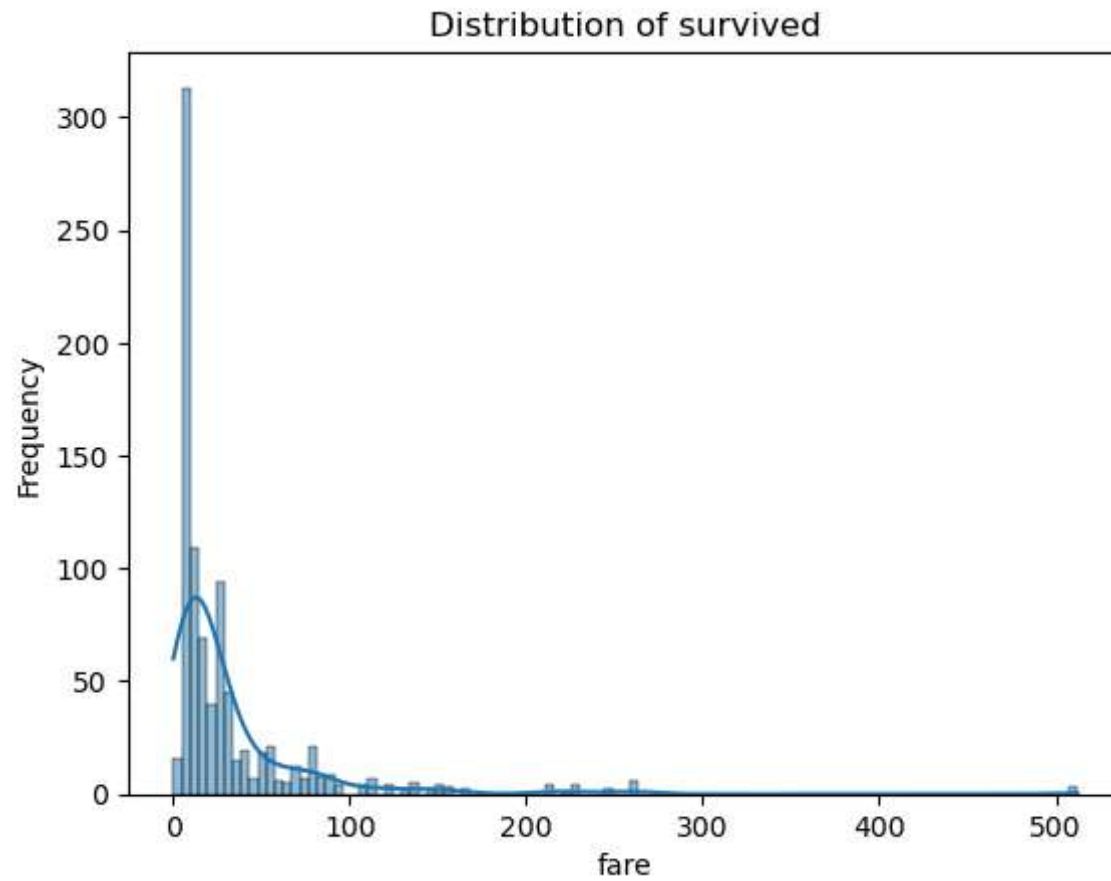
```
Out[48]: survived      0.364636  
pclass      -0.495610  
age         0.447515  
sibsp       3.051088  
parch       2.628645  
fare        4.583206  
adult_male  -0.311181  
alone       -0.274151  
dtype: float64
```

```
In [49]: numerical_cols = df2.select_dtypes(include='number').columns  
numerical_cols
```

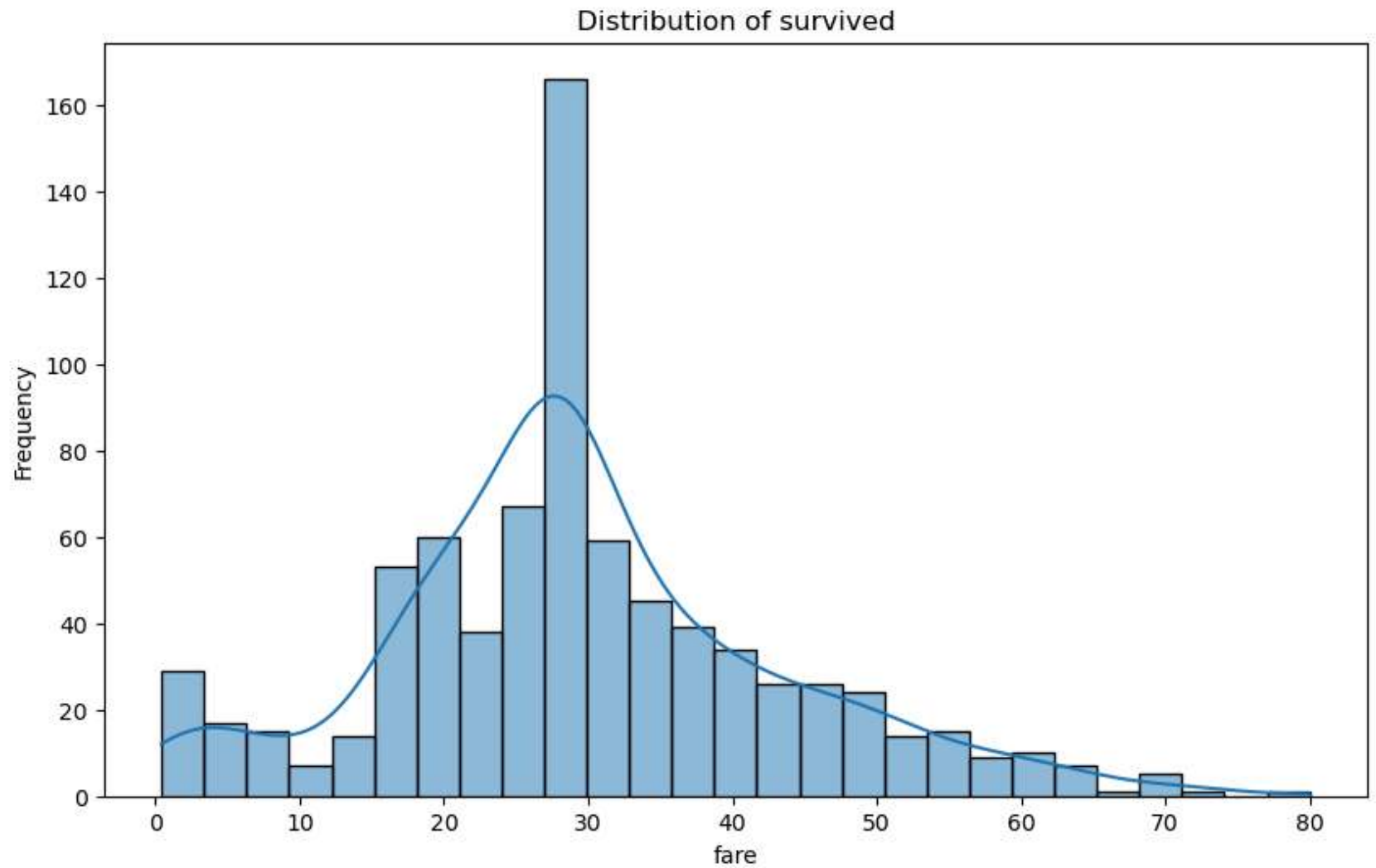
```
Out[49]: Index(['survived', 'pclass', 'age', 'sibsp', 'parch', 'fare'], dtype='object')
```

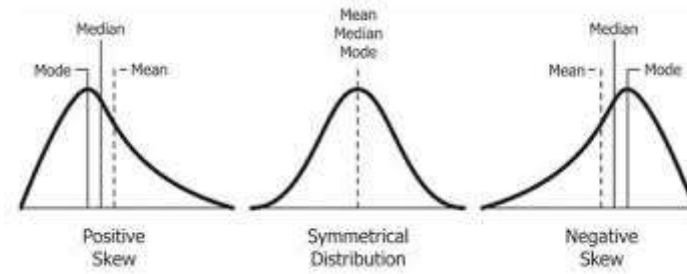
In [98]:

```
sns.histplot(df["fare"], kde=True)
plt.title("Distribution of survived")
plt.xlabel("fare")
plt.ylabel('Frequency')
plt.show()
```



```
In [51]: plt.figure(figsize=(10, 6))
sns.histplot(df2["age"], kde=True)
plt.title("Distribution of survived")
plt.xlabel("fare")
plt.ylabel('Frequency')
plt.show()
```





mean>median>mode===positively

```
In [52]: numerical_cols
```

```
Out[52]: Index(['survived', 'pclass', 'age', 'sibsp', 'parch', 'fare'], dtype='object')
```

```
In [53]: for i in numerical_cols:
plt.figure(figsize=(10, 6))
sns.histplot(df2[i].dropna(), kde=True)
plt.xlabel(i)
plt.ylabel('Frequency')
plt.show()
```



```
In [54]: df2.skew()
```

C:\Users\HP\AppData\Local\Temp\ipykernel_9652\834593052.py:1: FutureWarning: Dropping of nuisance columns in DataFrame reductions (with 'numeric_only=None') is deprecated; in a future version this will raise TypeError. Select only valid columns before calling the reduction.
df2.skew()

```
Out[54]: survived      0.364636  
pclass      -0.495610  
age         0.447515  
sibsp       3.051088  
parch       2.628645  
fare        4.583206  
adult_male  -0.311181  
alone       -0.274151  
dtype: float64
```

```
In [100]: # detecting outlier  
### method 1 -iqr  
df.describe()
```

Out[100]:

	survived	pclass	age	sibsp	parch	fare
count	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

```
In [55]: #fare column  
df2["fare"].skew()
```

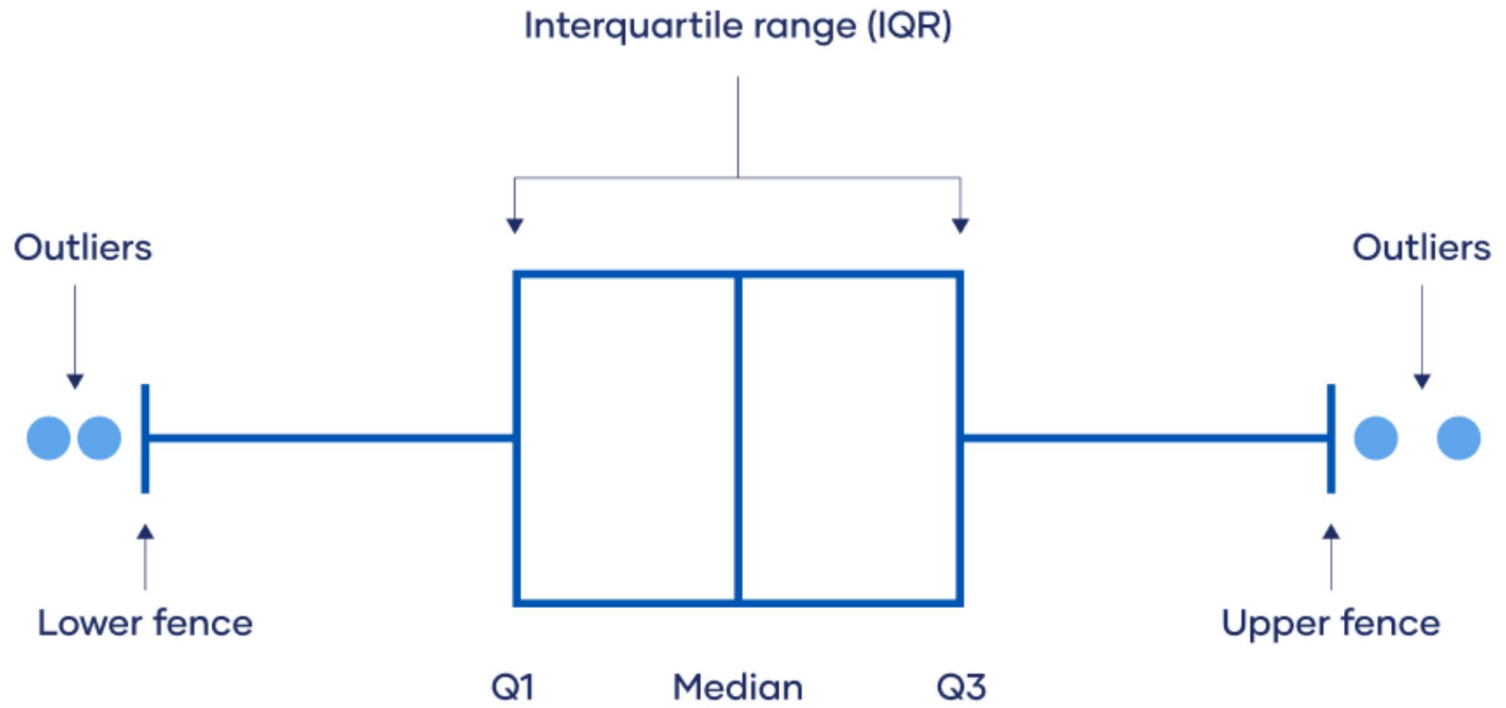
Out[55]: 4.583205969233933

```
In [101]: # using IQR
q1=df.fare.quantile(0.25)
print("q1 = ",q1)
q3=df.fare.quantile(0.75)
print("q3 = ",q3)
IQR=q3-q1
print("IQR = ",IQR)

q1 = 7.9104
q3 = 31.0
IQR = 23.0896
```

```
In [57]: df2["fare"].describe()
```

```
Out[57]: count    782.000000
mean         34.595913
std          52.176458
min           0.000000
25%           8.050000
50%          15.875000
75%          33.375000
max          512.329200
Name: fare, dtype: float64
```





$$\text{Interquartile Range} = Q3 - Q1$$

Outliers are values that are less than: $Q1 - (1.5 \times \text{IQR})$

Outliers are values that are more than: $Q3 + (1.5 \times \text{IQR})$

E.g. if: Minimum = 5 $Q1 = 10$ $Q2 = 18$ $Q3 = 20$ Maximum = 25

$$\text{Interquartile Range} = Q3 - Q1$$

$$\text{Interquartile Range} = 20 - 10 = 10$$

Lower fence: $Q1 - (1.5 \times \text{IQR})$ Upper fence: $Q3 + (1.5 \times \text{IQR})$

Lower fence: $10 - (1.5 \times 10)$ Upper fence: $20 + (1.5 \times 10)$

Lower fence: -5 Upper fence: 35

Any value less than -5 or larger than 35 is an outlier

```
In [58]: lower_whisker=q1-1.5*IQR  
         upper_whisker=q3+1.5*IQR  
         lower_whisker,upper_whisker
```

```
Out[58]: (-29.937499999999996, 71.3625)
```

```
In [59]: df2[(df2.fare<lower_whisker) | (df2.fare>upper_whisker)] #outliers
```

Out[59]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	embark_town	alive	alone
27	0	1	male	19.0	3	2	263.0000	S	First	man	True	Southampton	no	False
31	1	1	female	28.0	1	0	146.5208	C	First	woman	False	Cherbourg	yes	False
34	0	1	male	28.0	1	0	82.1708	C	First	man	True	Cherbourg	no	False
52	1	1	female	49.0	1	0	76.7292	C	First	woman	False	Cherbourg	yes	False
62	0	1	male	45.0	1	0	83.4750	S	First	man	True	Southampton	no	False
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
820	1	1	female	52.0	1	1	93.5000	S	First	woman	False	Southampton	yes	False
835	1	1	female	39.0	1	1	83.1583	C	First	woman	False	Cherbourg	yes	False
849	1	1	female	28.0	1	0	89.1042	C	First	woman	False	Cherbourg	yes	False
856	1	1	female	45.0	1	1	164.8667	S	First	woman	False	Southampton	yes	False
879	1	1	female	56.0	0	1	83.1583	C	First	woman	False	Cherbourg	yes	False

100 rows × 14 columns

In [60]: df2

Out[60]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	Southampton	no	True
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
885	0	3	female	39.0	0	5	29.1250	Q	Third	woman	False	Queenstown	no	False
887	1	1	female	19.0	0	0	30.0000	S	First	woman	False	Southampton	yes	True
888	0	3	female	28.0	1	2	23.4500	S	Third	woman	False	Southampton	no	False
889	1	1	male	26.0	0	0	30.0000	C	First	man	True	Cherbourg	yes	True
890	0	3	male	32.0	0	0	7.7500	Q	Third	man	True	Queenstown	no	True

782 rows × 14 columns

In [120]: `import pandas as pd`

```
Q1 = df['fare'].quantile(0.25)
Q3 = df['fare'].quantile(0.75)
IQR = Q3 - Q1
```

```
lower_whisker = Q1 - .5 * IQR
upper_whisker = Q3 + .5 * IQR
```

```
# Filter to get DataFrame without outliers
```

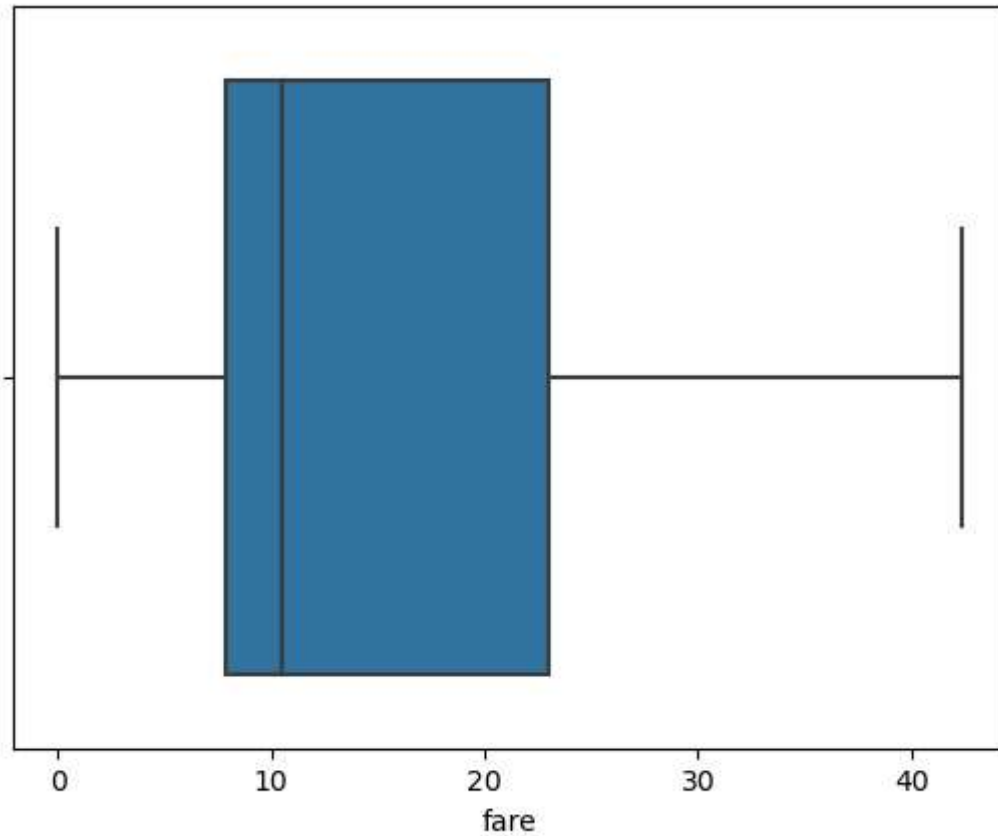
```
df_without_outlier = df[(df['fare'] >= lower_whisker) & (df['fare'] <= upper_whisker)]
```



```
In [121]: sns.boxplot(df_without_outlier["fare"])  
plt.show()
```

C:\Users\HP\anaconda3\lib\site-packages\seaborn_decorators.py:36: FutureWarning: Pass the following variable as a keyword arg: x. From version 0.12, the only valid positional argument will be `data`, and passing other arguments without an explicit keyword will result in an error or misinterpretation.

warnings.warn(



```
In [122]: df2_without_outlier.skew()
```

C:\Users\HP\AppData\Local\Temp\ipykernel_9652\676484663.py:1: FutureWarning: Dropping of nuisance columns in DataFrame reductions (with 'numeric_only=None') is deprecated; in a future version this will raise TypeError. Select only valid columns before calling the reduction.

```
df2_without_outlier.skew()
```

```
Out[122]: survived      0.561605  
pclass      -0.883982  
age         0.487560  
sibsp       3.170729  
parch       2.862404  
fare        1.390565  
adult_male  -0.439281  
alone       -0.439281  
dtype: float64
```

```
In [123]: # we can create a function for that
```

```
In [65]: column= df2.select_dtypes(include='number').columns  
column
```

```
Out[65]: Index(['survived', 'pclass', 'age', 'sibsp', 'parch', 'fare'], dtype='object')
```

```
In [66]: import pandas as pd

def remove_outliers(df, columns):
    df_filtered = df.copy() # Make a copy of the original DataFrame

    for col in columns:
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1

        lower_whisker = Q1 - 1.5 * IQR
        upper_whisker = Q3 + 1.5 * IQR

        # Filter out rows where column values are outliers
        df_filtered = df_filtered[(df_filtered[col] >= lower_whisker) & (df_filtered[col] <= upper_whisker)]

    return df_filtered
```

```
In [67]: dff=remove_outliers(df2,["fare","sibsp","parch"])
```

```
In [68]: dff
```

```
Out[68]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	Southampton	no	True
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
883	0	2	male	28.0	0	0	10.5000	S	Second	man	True	Southampton	no	True
887	1	1	female	19.0	0	0	30.0000	S	First	woman	False	Southampton	yes	True
888	0	3	female	28.0	1	2	23.4500	S	Third	woman	False	Southampton	no	False
889	1	1	male	26.0	0	0	30.0000	C	First	man	True	Cherbourg	yes	True
890	0	3	male	32.0	0	0	7.7500	Q	Third	man	True	Queenstown	no	True

632 rows × 14 columns

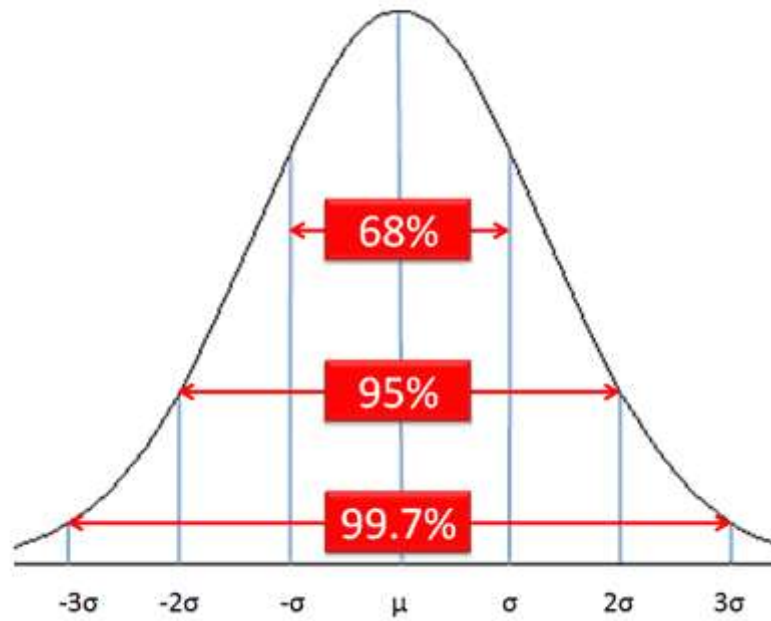
```
In [69]: dff.skew()
```

C:\Users\HP\AppData\Local\Temp\ipykernel_9652\2555945417.py:1: FutureWarning: Dropping of nuisance columns in DataFrame reductions (with 'numeric_only=None') is deprecated; in a future version this will raise TypeError. Select only valid columns before calling the reduction.

```
dff.skew()
```

```
Out[69]: survived      0.496738
pclass      -0.780698
age         0.620454
sibsp       1.538572
parch       2.168785
fare        1.560618
adult_male  -0.610094
alone       -0.653974
dtype: float64
```

In [124]: `#method 2: z score`

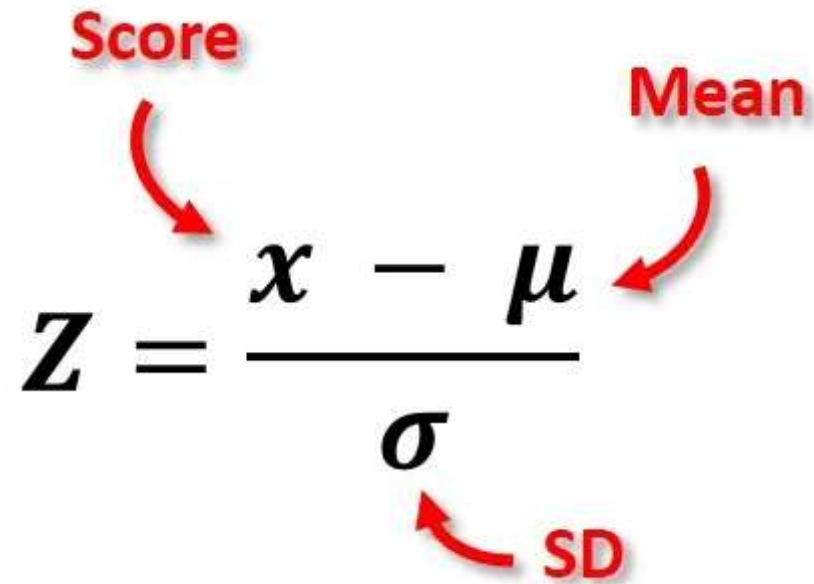


Score

Mean

$$Z = \frac{x - \mu}{\sigma}$$

SD

A diagram showing the Z-score formula. The formula is $Z = \frac{x - \mu}{\sigma}$. Three red arrows point from labels to parts of the formula: one from 'Score' to 'Z', one from 'Mean' to ' μ ', and one from 'SD' to ' σ '.

```
In [128]: x=df["fare"]  
x
```

```
Out[128]: 0      7.2500  
          1     71.2833  
          2      7.9250  
          3     53.1000  
          4      8.0500  
          ...  
          886    13.0000  
          887    30.0000  
          888    23.4500  
          889    30.0000  
          890     7.7500  
          Name: fare, Length: 891, dtype: float64
```

```
In [72]: mean=df2["fare"].mean()  
mean
```

```
Out[72]: 34.59591291560101
```

```
In [73]: sd=df2["fare"].std()  
sd
```

```
Out[73]: 52.17645834257139
```

```
In [134]: df["zscore"]=(x-mean)/sd
```

```
In [135]: df
```

```
Out[135]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone	
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False	-(
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False	(
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True	-(
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False	(
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True	-(
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
886	0	2	male	27.0	0	0	13.0000	S	Second	man	True	NaN	Southampton	no	True	-(
887	1	1	female	19.0	0	0	30.0000	S	First	woman	False	B	Southampton	yes	True	-(
888	0	3	female	NaN	1	2	23.4500	S	Third	woman	False	NaN	Southampton	no	False	-(
889	1	1	male	26.0	0	0	30.0000	C	First	man	True	C	Cherbourg	yes	True	-(
890	0	3	male	32.0	0	0	7.7500	Q	Third	man	True	NaN	Queenstown	no	True	-(

891 rows × 16 columns



```
In [150]: df2_cleaned = df[(df['zscore'] >= -.5) & (df['zscore'] <= .5)]
df2_cleaned
```

Out[150]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone	
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False	(
6	0	1	male	54.0	0	0	51.8625	S	First	man	True	E	Southampton	no	True	(
7	0	3	male	2.0	3	1	21.0750	S	Third	child	False	NaN	Southampton	no	False	-(
8	1	3	female	27.0	0	2	11.1333	S	Third	woman	False	NaN	Southampton	yes	False	-(
9	1	2	female	14.0	1	0	30.0708	C	Second	child	False	NaN	Cherbourg	yes	False	-(
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
885	0	3	female	39.0	0	5	29.1250	Q	Third	woman	False	NaN	Queenstown	no	False	-(
886	0	2	male	27.0	0	0	13.0000	S	Second	man	True	NaN	Southampton	no	True	-(
887	1	1	female	19.0	0	0	30.0000	S	First	woman	False	B	Southampton	yes	True	-(
888	0	3	female	NaN	1	2	23.4500	S	Third	woman	False	NaN	Southampton	no	False	-(
889	1	1	male	26.0	0	0	30.0000	C	First	man	True	C	Cherbourg	yes	True	-(

476 rows × 17 columns



```
In [151]: df2_cleaned["fare"].skew()
```

Out[151]: 1.0231297514054218


```
In [152]: #METHOD 3
df2
```

Out[152]:

	survived	pclass	sex	age	sibsp	parch	embarked	class	who	adult_male	embark_town	alive	alone	zscore	log_fare
0	0	3	male	22.0	1	0	S	Third	man	True	Southampton	no	False	-0.524104	2.110213
1	1	1	female	38.0	1	0	C	First	woman	False	Cherbourg	yes	False	0.703141	4.280593
2	1	3	female	26.0	0	0	S	Third	woman	False	Southampton	yes	True	-0.511168	2.188856
3	1	1	female	35.0	1	0	S	First	woman	False	Southampton	yes	False	0.354644	3.990834
4	0	3	male	35.0	0	0	S	Third	man	True	Southampton	no	True	-0.508772	2.202765
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
885	0	3	female	39.0	0	5	Q	Third	woman	False	Queenstown	no	False	-0.104854	3.405355
887	1	1	female	19.0	0	0	S	First	woman	False	Southampton	yes	True	-0.088084	3.433987
888	0	3	female	28.0	1	2	S	Third	woman	False	Southampton	no	False	-0.213620	3.196630
889	1	1	male	26.0	0	0	C	First	man	True	Cherbourg	yes	True	-0.088084	3.433987
890	0	3	male	32.0	0	0	Q	Third	man	True	Queenstown	no	True	-0.514522	2.169054

782 rows × 15 columns



```
In [157]: x=np.log1p(df["fare"])
x.skew()
```

Out[157]: 0.3949280095189306

```
In [154]: df["fare"].min()
```

Out[154]: 0.0

```
In [155]: df['log_fare'] = np.log1p(df['fare'])
df
```

Out[155]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone	
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False	-(
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False	(
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True	-(
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False	(
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True	-(
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
886	0	2	male	27.0	0	0	13.0000	S	Second	man	True	NaN	Southampton	no	True	-(
887	1	1	female	19.0	0	0	30.0000	S	First	woman	False	B	Southampton	yes	True	-(
888	0	3	female	NaN	1	2	23.4500	S	Third	woman	False	NaN	Southampton	no	False	-(
889	1	1	male	26.0	0	0	30.0000	C	First	man	True	C	Cherbourg	yes	True	-(
890	0	3	male	32.0	0	0	7.7500	Q	Third	man	True	NaN	Queenstown	no	True	-(

891 rows × 17 columns



```
In [156]: df["log_fare"].skew()
```

Out[156]: 0.3949280095189306

```
In [159]: df.drop(["fare"],axis=1,inplace=True)
```

```
In [160]: df
```

```
Out[160]:
```

	survived	pclass	sex	age	sibsp	parch	embarked	class	who	adult_male	deck	embark_town	alive	alone	zscore
0	0	3	male	22.0	1	0	S	Third	man	True	NaN	Southampton	no	False	-0.524104
1	1	1	female	38.0	1	0	C	First	woman	False	C	Cherbourg	yes	False	0.703141
2	1	3	female	26.0	0	0	S	Third	woman	False	NaN	Southampton	yes	True	-0.511168
3	1	1	female	35.0	1	0	S	First	woman	False	C	Southampton	yes	False	0.354644
4	0	3	male	35.0	0	0	S	Third	man	True	NaN	Southampton	no	True	-0.508772
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
886	0	2	male	27.0	0	0	S	Second	man	True	NaN	Southampton	no	True	-0.413901
887	1	1	female	19.0	0	0	S	First	woman	False	B	Southampton	yes	True	-0.088084
888	0	3	female	NaN	1	2	S	Third	woman	False	NaN	Southampton	no	False	-0.213620
889	1	1	male	26.0	0	0	C	First	man	True	C	Cherbourg	yes	True	-0.088084
890	0	3	male	32.0	0	0	Q	Third	man	True	NaN	Queenstown	no	True	-0.514522

891 rows × 16 columns



```
In [161]: df.skew()
```

C:\Users\HP\AppData\Local\Temp\ipykernel_9652\1665899112.py:1: FutureWarning: Dropping of nuisance columns in DataFrame reductions (with 'numeric_only=None') is deprecated; in a future version this will raise TypeError. Select only valid columns before calling the reduction.

```
df.skew()
```

```
Out[161]: survived      0.478523
pclass      -0.630548
age         0.389108
sibsp       3.695352
parch       2.749117
adult_male  -0.420431
alone      -0.420431
zscore      4.787317
log_fare    0.394928
dtype: float64
```

In []:

In [165]: df

Out[165]:

	survived	pclass	sex	age	sibsp	parch	embarked	class	who	adult_male	deck	embark_town	alive	alone	zscore
0	0	3	male	22.0	1	0	S	Third	man	True	NaN	Southampton	no	False	-0.524104
1	1	1	female	38.0	1	0	C	First	woman	False	C	Cherbourg	yes	False	0.703141
2	1	3	female	26.0	0	0	S	Third	woman	False	NaN	Southampton	yes	True	-0.511168
3	1	1	female	35.0	1	0	S	First	woman	False	C	Southampton	yes	False	0.354644
4	0	3	male	35.0	0	0	S	Third	man	True	NaN	Southampton	no	True	-0.508772
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
886	0	2	male	27.0	0	0	S	Second	man	True	NaN	Southampton	no	True	-0.413901
887	1	1	female	19.0	0	0	S	First	woman	False	B	Southampton	yes	True	-0.088084
888	0	3	female	NaN	1	2	S	Third	woman	False	NaN	Southampton	no	False	-0.213620
889	1	1	male	26.0	0	0	C	First	man	True	C	Cherbourg	yes	True	-0.088084
890	0	3	male	32.0	0	0	Q	Third	man	True	NaN	Queenstown	no	True	-0.514522

891 rows × 16 columns



In [166]: df["log_fare"].isnull().sum()

Out[166]: 0

```
In [167]: df.skew()
```

```
C:\Users\HP\AppData\Local\Temp\ipykernel_9652\1665899112.py:1: FutureWarning: Dropping of nuisance columns in DataFrame reductions (with 'numeric_only=None') is deprecated; in a future version this will raise TypeError. Select only valid columns before calling the reduction.
```

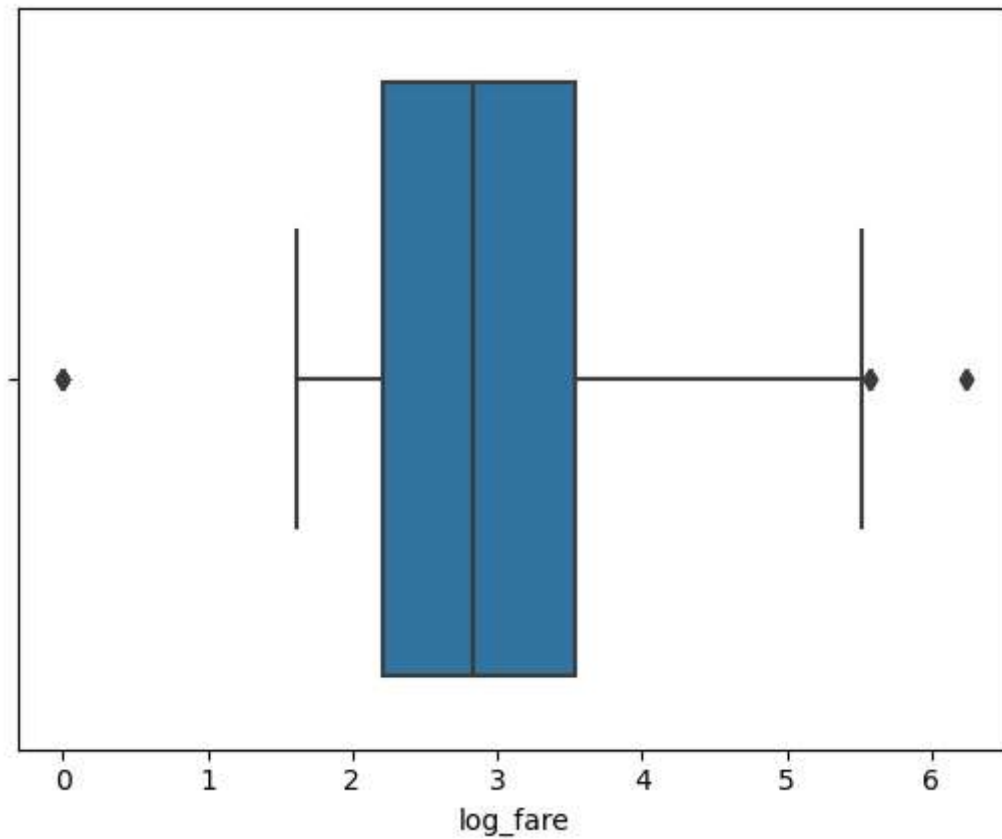
```
df.skew()
```

```
Out[167]: survived      0.478523  
pclass      -0.630548  
age         0.389108  
sibsp       3.695352  
parch       2.749117  
adult_male  -0.420431  
alone       -0.420431  
zscore      4.787317  
log_fare    0.394928  
dtype: float64
```

```
In [90]: sns.boxplot(df2["log_fare"])
```

C:\Users\HP\anaconda3\lib\site-packages\seaborn_decorators.py:36: FutureWarning: Pass the following variable as a keyword arg: x. From version 0.12, the only valid positional argument will be `data`, and passing other arguments without an explicit keyword will result in an error or misinterpretation.
warnings.warn(

```
Out[90]: <AxesSubplot:xlabel='log_fare'>
```



```
In [ ]:
```

